Model Predictive Control and Reinforcement Learning

Exercise Sheet 7: Designing an OCP with `acados`

Joschka Boedecker, Moritz Diehl, Jonathan Frey, Katrin Baumgärtner, Leonard Fichtner, Jasper Hoffmann

Exercise 1: Getting started with `acados`

In this exercise, you will implement a simple MPC controller for the cart pole system (aka inverted pendulum on a cart), see Figure 1. The goal is to implement a controller that balances the pole in the closed-loop simulation provided in `example_closed_loop.py`.

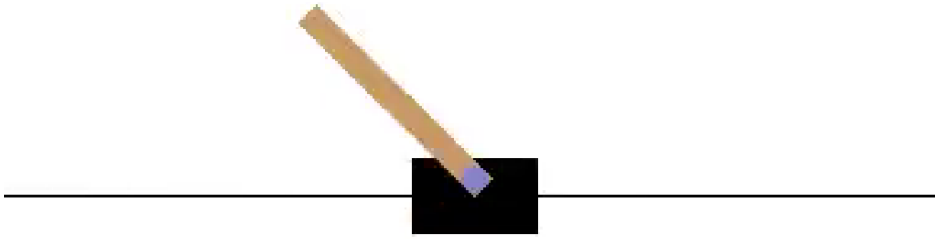


Figure 1: An illustration of cart pole while swinging up.

1.1 Familiarize yourself with the given template. The script creates an `acados` OCP solver and an `acados` integrator which are run in a closed-loop simulation.

1.2 Complete the controller formulation in the function `setup_ocp` provided in `setup_ocp.py`. For a given initial state s, this function formulates a simple MPC controller described by the problem formulation

$$\begin{aligned} \min_{x_0,u_0,x_1,\dots,u_{N-1},x_N} \quad & \frac{1}{N}\sum_{k=0}^{N-1} l(x_k,u_k) + \overline{V}(x_N) \\ \text{s.t. } \quad & x_0 = s \\ & x_{k+1} = f^{\text{dyn}}(x_k,u_k),\ k=0,\dots,N-1 \\ & \underline{u} \le u_k \le \overline{u},\ k=0,\dots,N-1, \end{aligned}$$

where $f^{\text{dyn}}(x_k,u_k)$ are the dynamics of the cart pole system, given by one step of an RK4 integrator, $\underline{u}$ and $\overline{u}$ are the actuator limits, and the stage costs and terminal cost are given by

$$l(x_k,u_k) = \frac{1}{2}|| \begin{pmatrix} x_k \\ u_k \end{pmatrix} - \begin{pmatrix} x^{\text{ref}} \\ u^{\text{ref}} \end{pmatrix} ||^2_W$$

and

$$\overline{V}(x_N) = \frac{1}{2}||x_N - x_{\text{e}}^{\text{ref}}||^2_{W_{\text{e}}}.$$

Set the values of $x^{\text{ref}}, u^{\text{ref}}, x_{\text{e}}^{\text{ref}}, W, W_{\text{e}}$ in a way that the controller successfully balances the pole. This will be indicated by messages printed by `example_closed_loop.py`.

1.3 Currently, both the integrator and the controller use the same model as `AcadosSim.from_ocp(sim)` creates an initial value problem (IVP) description with the same model and integration options as defined in the OCP description. Increase the mass of the cart within the integrator by 10% to introduce model-plant mismatch. How does the reward change compared to the simulation without model-plant mismatch?

1.4 Try to reduce the computation time required by the controller. There are many ways to achieve this. Checkout the solver options, `ocp.solver_options`. Try changing the (QP) solver tolerances, use real-time iterations, a nonuniform discretization grid, integration settings within the OCP, partial condensing, ...

For your new settings, write down the total reward and the fraction of the computation required by your solver wrt. the original setup for comparison with your colleagues.